



PB-FED:

Password-Based File Encryption- Decryption System

A Lightweight Python-Based Offline Security Tool

Abstract: Enhancing Data Confidentiality

Importance of Data Confidentiality

Addressing the critical need for data confidentiality in digital communication.

PB-FED: A Versatile Tool

A Python-based tool for encrypting and decrypting various file types including text, images, and binary files.

Key Features

- Uses password-derived keys via SHA-256 + Base64, with Fernet (AES-256 + HMAC) for robust encryption.
- Available in both console and GUI versions.
- Operates entirely offline, ensuring maximum privacy.

User-Friendly Design

Lightweight, user-friendly, and ideal for personal and academic use due to its simplicity and effectiveness.

The Urgent Need for Encryption



Rising Threats

The digital landscape faces escalating threats from data breaches, unauthorized access, and sophisticated cyberattacks.

Essential for All

Data protection is crucial for individuals, students, educators, and small organizations alike.

Accessibility Gap

Many users lack access to secure, yet easy-to-use, encryption tools that meet their specific needs.

Challenges with Existing Encryption Tools

VeraCrypt, BitLocker, AxCrypt

While secure, these tools are often heavy, GUI-dependent, and require complex installation or administrative privileges.

Offline & User-Friendliness

Most tools are not fully offline, compromising privacy, and lack beginner-friendliness.

Limited Scope

Many do not support both text and image encryption within a single, unified interface.

Motivation Behind PB-FED: Bridging the Gap

Lightweight & Offline Focus

A core need for a lightweight, entirely offline, and simple encryption tool.

Unified Data Support

Crucially, the tool must support both text and image encryption seamlessly, addressing a common user requirement.

Cryptographic Abstraction

Designed to be used without requiring in-depth cryptographic knowledge, making security accessible.

Platform Independence

Ensuring compatibility across various operating systems, especially beneficial in resource-constrained environments.

Existing Encryption Tools & Their Limitations

VeraCrypt	Full Disk Encryption	Complex UI, installation required, not offline-first
BitLocker	OS-Level Encryption	Needs admin rights, not cross-platform, vendor-locked
AxCrypt	File Encryption	GUI-heavy, Windows-focused, limited offline support

While these tools offer strong encryption, they often require administrative access, significant system integration, and are not designed for lightweight, beginner-friendly, or fully offline usage.

Python in Cryptography: A Growing Trend



Python is increasingly becoming a preferred language for educational and practical cryptographic applications.

- **cryptography:** Robust, high-level interface for cryptographic primitives.
- **hashlib:** Supports secure hashing algorithms like SHA-256.
- **base64:** Used for efficient key encoding and data representation.
- **tkinter:** Facilitates the creation of simple graphical user interfaces.

These libraries empower developers to build real-world encryption systems with simplicity and effectiveness, as demonstrated by PB-FED.

Identified Gaps & PB-FED's Solution

1

Lack of Unified CLI Tools

Few open-source command-line tools support both text and image encryption comprehensively.

2

Offline & Key Management Deficiencies

Most tools lack robust offline capabilities and intuitive password-based key derivation without complex file management.

3

Limited File Type Support

Many educational tools focus solely on text, omitting crucial support for binary data like PDFs, images, and media files.

4

PB-FED: The Comprehensive Answer

Combines ease-of-use, security, and offline access for text, image, and PDF encryption, requiring only a Python installation.

What is PB-FED?

PB-FED is a Password-Based File Encryption-Decryption System, offering robust security for your data without the need for complex key management.

- Python-based, offline, and lightweight tool for personal data protection.
- Encrypts and decrypts text, image, and binary files using password-derived cryptographic keys.
- Employs Fernet (AES-256-CBC + HMAC) for secure symmetric encryption.
- Ideal for educational use and low-resource environments.

Key Features of PB-FED



Password-Based Security

No need to store cryptographic keys; derived on-the-fly.



Multi-file Support

Encrypt/decrypt text, images, PDFs, and binary files.



Modular & Lightweight

Console-based with a simple GUI for ease of use.



Offline Operation

Entirely local, ensuring privacy with no internet dependency.



Cross-platform Compatibility

Runs seamlessly on Windows, Linux, and macOS.



Educational Utility

Great for learning real-world cryptography concepts.

Supported Data Formats

PB-FED is versatile, supporting various data types for encryption and decryption, making it a flexible tool for diverse security needs.

✔ Plaintext Messages	Input directly via terminal for quick encryption.
✔ Text Files (.txt)	Securely read and write encrypted text files.
✔ Image Files (.jpg, .png)	Encrypts image files as binary data, preserving visual integrity.
✔ PDF and Binary Files	Handles .pdf, .bin, and other binary formats with Fernet.

PB-FED System Architecture

PB-FED employs a robust 3-layer architecture for efficient and secure file processing, ensuring clear separation of concerns.



User Interface (UI) Layer

Built with tkinter for GUI, presenting menus and collecting inputs.



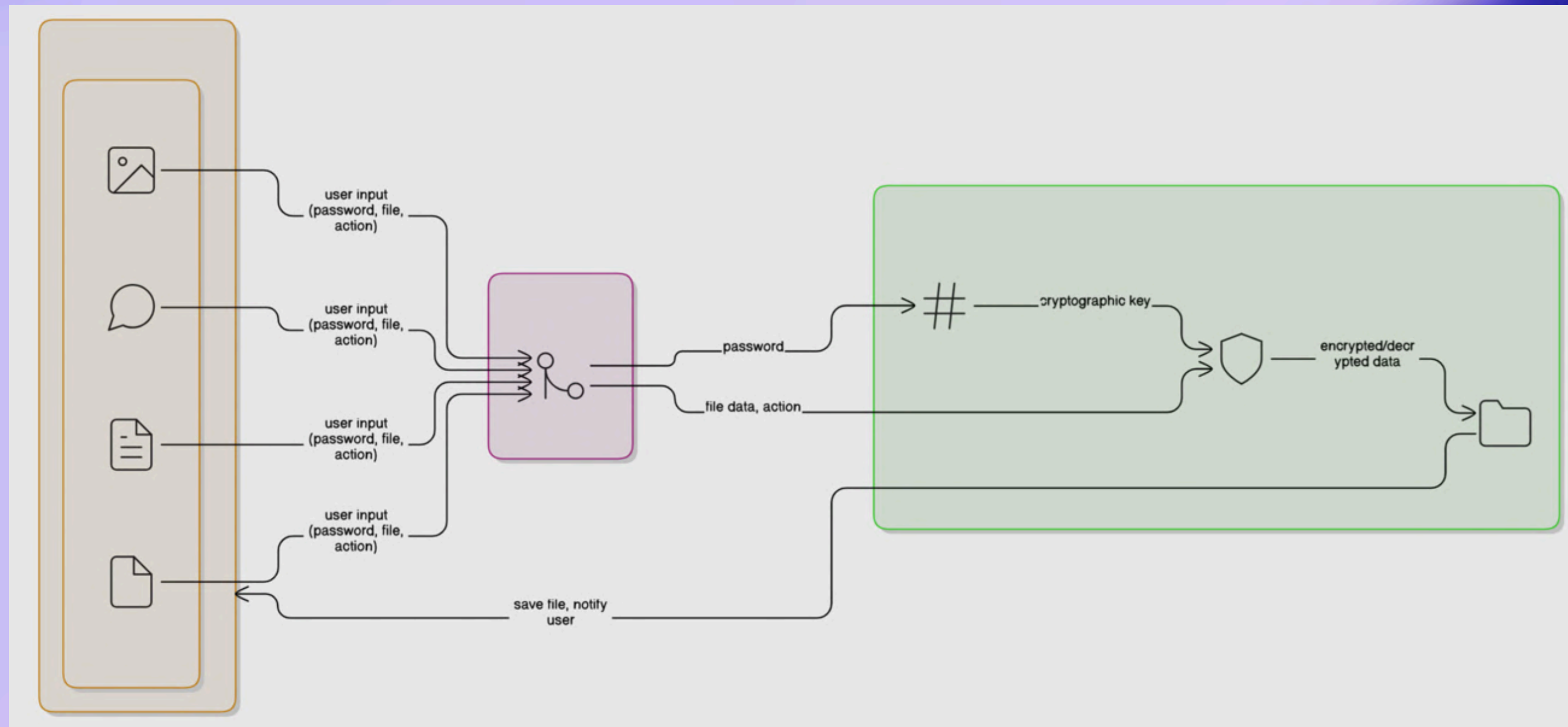
Application Logic Layer

Acts as a controller, routing user inputs and validating formats.



Encryption-Decryption Engine (Backend)

Performs SHA-256 hashing, Base64 encoding, and Fernet operations.



PB-FED User Flow (Console Mode)

1

Main Menu

Select operation: Encrypt or Decrypt.

2

Select Data Type

Choose between Text, Image, or Binary.

3

Enter Password

Provide password for key derivation.

4

Input Path/Text

Specify file path or directly input text.

5

Output Saved

Encrypted/decrypted file generated.

Key Derivation & Encryption Process



Password to Key Derivation:

- User password converted to bytes.
- SHA-256 hash generates 256-bit digest.
- Base64 encoded to a 32-byte Fernet key.



Encryption & Decryption (Fernet):

- Fernet uses AES-256 in CBC mode and HMAC for integrity.
- Encrypted data includes ciphertext, timestamp, and HMAC.
- Same password regenerates key for decryption.



Why this is secure:

- Prevents direct use of raw passwords.
- One-way hashing resists reverse-engineering.
- Keys are never stored—always derived on-the-fly.



Advantages:

- Data integrity verification using HMAC.
- Authenticated encryption for confidentiality and authenticity.
- No separate IV/key handling needed.

Password Based Key Derivation

PBKD is the foundation of our encryption system, ensuring that sensitive data is protected through a robust, one-way process that transforms user passwords into secure cryptographic keys. This method prioritizes security by preventing direct storage of passwords.



Symmetric Encryption with Fernet

Fernet, part of Python's cryptography library, provides robust symmetric encryption crucial for PB-FED's security.

Built on Strong Primitives

Fernet combines AES in CBC mode (128-bit) with HMAC (SHA-256) for both confidentiality and integrity.

Simplified Key Management

The PB-FED system derives a secure key from your password, eliminating the need to store sensitive keys.

Authenticated Encryption

Each encrypted output includes ciphertext, a timestamp, and an HMAC, ensuring both data privacy and authenticity.

Symmetric Encryption with Fernet

1

What is Fernet?

Part of Python's cryptography library, Fernet combines AES in CBC mode (128-bit) and HMAC (SHA-256) for robust message authentication.

2

How it Works in PB-FED

A Fernet object is initialized with a Base64-encoded key. Plaintext or binary data is encrypted, producing ciphertext, a timestamp, and an HMAC.

3

Key Advantages

Fernet provides authenticated encryption, ensuring both confidentiality and data integrity. It simplifies key handling by wrapping IV and key management.

This method enhances data protection by ensuring that encrypted files are tamper-proof and securely linked to their original content.

Secure Decryption Process

Decryption Flow

The decryption process mirrors encryption: the user's password regenerates the Fernet key. This key is then used to load and decrypt the file, with HMAC verification ensuring integrity.

- User enters password
- Password converts to Fernet key
- Encrypted file is loaded
- HMAC verifies integrity
- Data is revealed if password matches

Incorrect Password Handling

If an incorrect password is provided, decryption is securely blocked. An error message is displayed without revealing any sensitive information or corrupting the data.

- ⊗ **No Data Compromise:** Incorrect passwords result in secure decryption failure, protecting data from unauthorized access.

Technologies and Libraries Used



Python 3.10+

The core programming language for the application.



hashlib

Used for SHA-256 password hashing.



base64

Key conversion to Fernet-compatible format.



cryptography.fernet

Symmetric encryption and decryption.



tkinter

For graphical user interface (GUI).



os & sys

Path handling and file management.

Input Validation and Output Flow

1

Input Types

- Text data (direct string encryption)
- Files (text, binary, optional images)
- User password (minimum length enforced)

2

Output Structure

- Encrypt files with **_encrypted** suffix as same extension as of provided file
- Decrypted files with **_decrypted** suffix
- Clear status messages via console or GUI

3

Validation & Security

- Invalid paths/file types trigger error prompts
- Wrong passwords result in secure failure
- Passwords are never logged; temporary data cleared

Testing

Test Case	Input Type	Status	Description
TC01	Plain text	✓ Passed	Encrypted & decrypted sample text
TC02	Text file (.txt)	✓ Passed	Verified output matches input
TC03	Binary file (.exe/.pdf)	✓ Passed	No corruption post decryption
TC04	Wrong password	✓ Passed	Decryption blocked with error
TC05	Large file (>50MB)	✓ Passed	Time-efficient encryption

Comprehensive testing confirms the reliability of key derivation, data integrity, and error handling across various scenarios and file types.

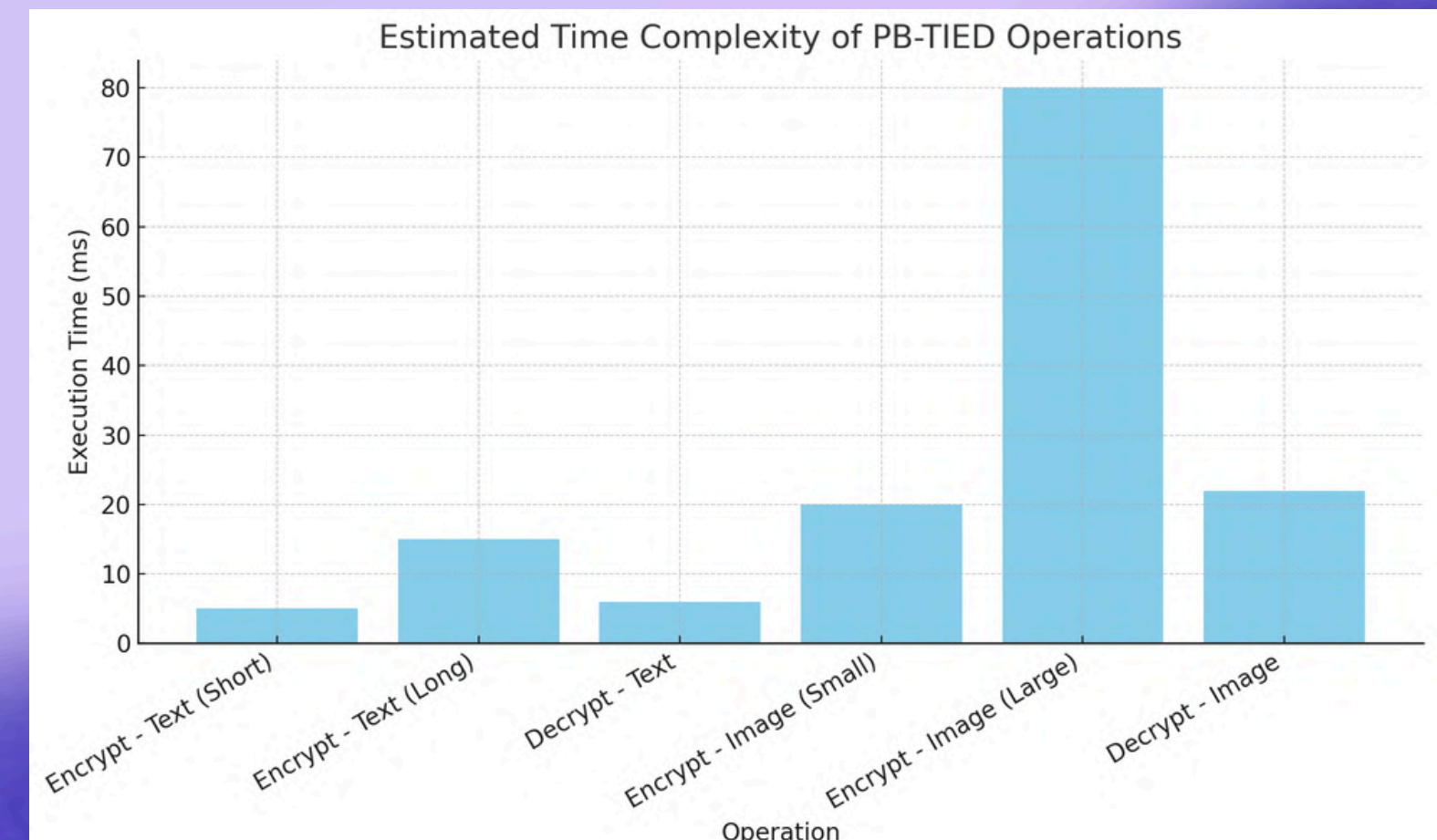
Results and Output Snapshots

Performance Observations: Encryption times averaged below 1 second for text/image files, and approximately 3-4 seconds for 100MB binary files, demonstrating efficient processing.

- Encryption time (avg): < 1s for text/image
- Encryption time (avg): ~1–1.5s for 100MB binary
- Decryption matches original content byte-for-byte
- Low memory usage during operation

User Experience: Both console and GUI interfaces provide clear prompts, masked password inputs, and fast feedback loops, ensuring a user-friendly experience.

- Easy to use interface (console or GUI)
- Clear prompts and error messages
- Fast feedback loop for success/failure



PB-FED performs secure, accurate, and reliable encryption/decryption, proving robust against incorrect passwords and unsupported files.

Conclusion & Future Scope

PB-FED is a lightweight, offline Python tool for secure encryption/ decryption of textual, image, and binary files. It uses SHA-256 for key derivation and Fernet for symmetric encryption, ensuring confidentiality, integrity, and authentication. Its offline capability suits air-gapped, personal, and educational uses, with a modular and accessible design.

Future Enhancements

1

UI/UX

Full GUI integration (tkinter/PyQt),
PDF/image previews.

2

Functionality

Multi-file/folder encryption, encryption
logs, cloud sync (opt-in),
steganography.

3

Security & Deployment

Advanced key derivation (PBKDF2,
bcrypt), cross-platform executables.

Thank You

